TP 3 : Unir et trouver deluxe

L'intégralité du TP est à réaliser en Ocaml.

Préliminaires

On a vu dans le cours la structure de données Unir et Trouver qui permet de représenter une partition d'ensemble. Elle est notamment utile pour connaître les différentes composantes connexes d'un graphe. On étudie ici une amélioration de cette structure.

Dans un graphe non orienté, un **pont** est une arête dont la suppression fait croître le nombre de composantes connexes. Lorsque l'on retire tous les ponts d'un graphe, les composantes connexes restantes sont appelées **blocs**. Par exemple, le graphe représenté figure 1 possède 4 ponts, reliant les paires de sommets (1, 3), (3, 4), (6, 7) et (11, 15) et 6 blocs, comme illustré figure 2. Dans ce problème, nous allons modifier Unir et Trouver pour permettre de déterminer les blocs d'un graphe non orienté décrit *en ligne*, c'est-à-dire en ajoutant les arêtes une par une. On suppose que l'on connaît à l'avance le nombre n de sommets du graphe, et que ceux-ci sont numérotés de 0 à $n-1$.

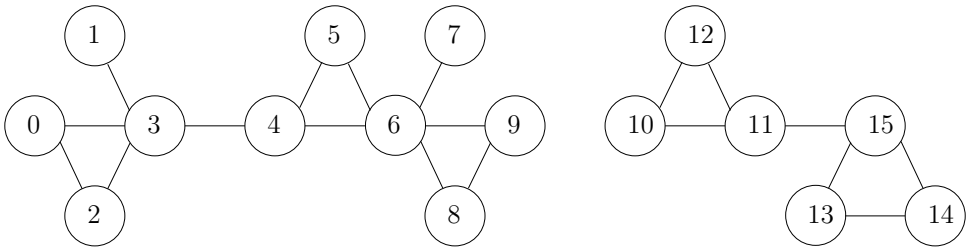


FIGURE 1 – Exemple de graphe

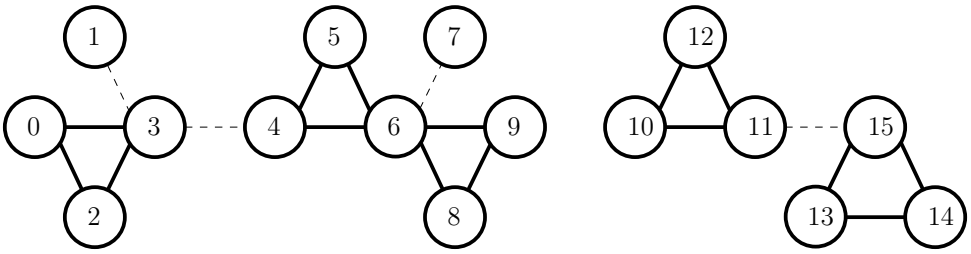


FIGURE 2 – Bloc en gras et ponts en hachuré

1 Représentation sous forme de forêt

Nous allons représenter le graphe sous forme d'une forêt à l'aide d'une structure de type Unir et Trouver et que nous appellerons but (pour block Unir et Trouver). Cette structure consiste en trois tableaux de taille le nombre n de sommets du graphe :

```
type but = { parent : int array; repr : bool array; rang : int array; }
```

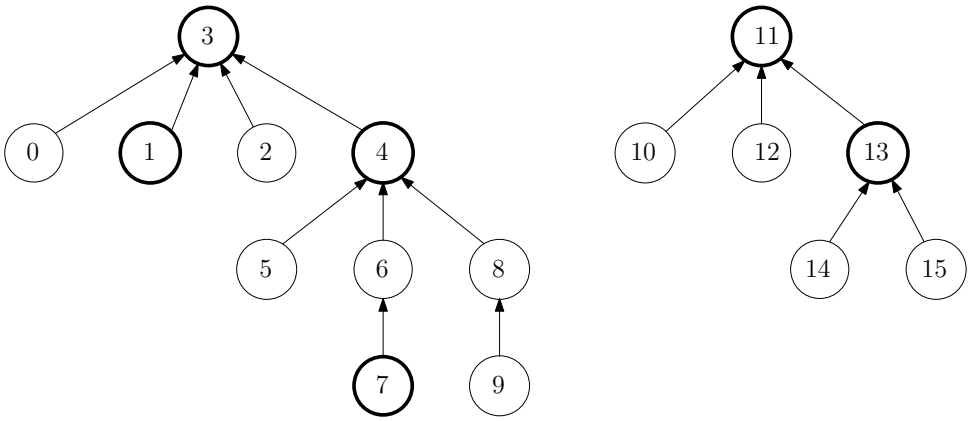
Chaque bloc a un unique représentant (défini de façon arbitraire), indiqué par le tableau `repr`. Chaque sommet a un parent, représenté par le tableau du même nom et qui a deux fonctions :

- si un sommet s est le représentant d'un bloc, notons-le b, alors soit `parent.(s)` $= s$, auquel cas s est racine d'un des arbres de la forêt, soit il indique un sommet d'un bloc b' tel qu'il existe un pont entre un sommet de b et un sommet de b' ;
- sinon, en suivant les parents successifs à partir d'un sommet s, on arrive au représentant de son bloc.

Tous les sommets d'une même composante connexe seront donc dans un même arbre.

Enfin, le tableau `rang` indique, pour chaque représentant de bloc, une mesure de la taille de ce bloc. Cette valeur interviendra uniquement à la question 8 lors de l'implémentation de l'opération de fusion entre blocs. Ainsi, tous les blocs sont représentés à la manière d'une structure de type Unir et Trouver, à la différence que les différents blocs

forment une forêt. On donne en figure 3 une représentation du graphe exemple de la figure 1 comme un élément de type but, ainsi que son illustration graphique.




```
{   parent = [|3; 3; 3; 3; 3; 4; 4; 6; 4; 8; 11; 11; 11; 11; 13; 13|];
    repr = [|false; true; false; true; true; false; false; true; false;
    false; false; true;false; true; false; false|];
    rang = [|0; 0; 0; 1; 2; 0; 0; 0; 0; 0; 0; 1; 0; 1; 0; 0|] }
```

FIGURE 3 – Structure but représentant le graphe de la figure 1.

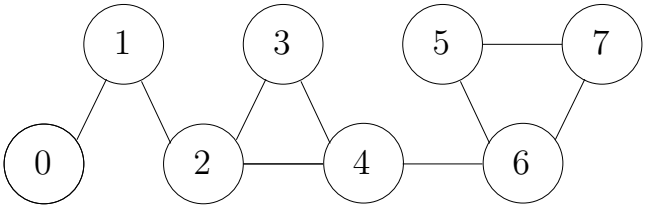


FIGURE 4 – Un autre graphe

Question 1 : Écrire une fonction `init : int -> but` qui, étant donné un entier n, renvoie une structure but décrivant un graphe de n sommets sans aucune arête : chaque sommet donne lieu à un bloc de rang 0 dont il est le représentant. Les valeurs du tableau `rang` seront initialisées à 0.

Question 2 : Écrire une fonction `find : but -> int -> int` qui renvoie le représentant dans la structure but du sommet passé en argument.

Question 3 : Écrire une fonction `find_compresse : int -> int -> int` qui renvoie le représentant dans la structure du sommet passé en paramètre. On mettra en œuvre la compression de chemin : on modifie les parents de tous les sommets croisés sur le chemin entre le sommet de départ et son représentant, chaque sommet ayant pour nouveau parent leur représentant commun.

Dans la suite, on utilisera systématiquement cette fonction pour trouver le représentant d'un sommet.

Question 4 : Écrire une fonction `blocs_array : but -> int list array` qui renvoie un tableau tel que `t.(i)` contient la liste des sommets dont i est le représentant. Ainsi, on pourra récupérer ses listes non vides pour avoir la liste des blocs. Par exemple, pour le graphe de la figure 1, représenté par la forêt de la figure 3, on pourra renvoyer :

```
[|[]; [1]; []; [3;2;0]; [9;8;6;5;4]; []; []; [7]; []; []; []; [12;11;10];
   []; [15;13;14]; []; []|]
```

L'ordre des sommets au sein des listes est sans importance.

Question 5 : Écrire une fonction `blocs : but -> int list list` qui renvoie la liste des blocs correspondant à

la structure passée en argument. L'ordre des blocs ainsi que l'ordre des sommets à l'intérieur d'un bloc n'est pas contraint. Ainsi, avec le graphe de la figure 1(représenté par la forêt de la figure 3), on peut avoir :

```
[[15; 13; 14]; [12; 11; 10]; [7]; [9; 8; 6; 5; 4]; [3; 2; 0]; [1]]
```

2 Ajout d'arêtes

Nous allons maintenant étudier l'effet de l'ajout d'une arête sur notre structure. Plusieurs cas sont possibles, suivant que les extrémités de l'arête ajoutée appartiennent à un même bloc, à des blocs distincts d'une même composante connexe, ou à des blocs de composantes connexes distinctes. Notons que si l'on ajoute une arête entre les sommets d'un même bloc, aucune modification n'est nécessaire. Pour traiter les deux autres cas, nous allons tout d'abord écrire quelques fonctions utilitaires avant d'implémenter la fonction d'ajout d'arête proprement dite.

Dans la suite, on appelle chaîne de représentants une suite finie non vide $(s_0, \ldots, s_p)$ de représentants telle que pour tout $i \in [\![0, p-1]\!]$, le sommet s_i est le représentant du parent de s_{i+1}. En particulier, les représentants apparaissent de la gauche vers la droite par profondeur croissante. De façon naturelle, une telle chaîne sera représentée par une liste.

Question 6 : Écrire une fonction `chaine_racine : but -> int -> int list` qui, étant donné un sommet s du graphe, renvoie la chaîne des représentants reliant le représentant de s à la racine de l'arbre correspondant. Ainsi, avec l'exemple précédent, la chaîne des représentants pour le sommet 5 est la liste `[3; 4]`. De même, pour le sommet 7 (qui est lui-même un représentant), on doit obtenir `[3; 4; 7]`.

2.1 Arêtes entre des sommets de composantes connexes distinctes

Lorsque l'on ajoute une arête entre deux sommets appartenant à des composantes connexes distinctes, cette nouvelle arête est un pont. En terme de structure but, si l'on note u et v les extrémités de l'arête ajoutée, on considère la chaîne des représentants allant de celui de l'un des sommets (supposons que l'on utilise u) vers la racine correspondante et on « retourne » toutes les flèches de la chaîne, faisant du représentant de u la nouvelle racine de son arbre. On change alors le parent du représentant de u pour le faire pointer vers celui de v.

La figure 5 illustre le résultat de l'ajout d'une arête entre les sommets 5 et 14. On a ici récupéré la chaîne des représentants jusqu'à la racine depuis 14 : `[13; 11]`. On a ensuite retourné les arêtes des représentants. 11 est ainsi devenu le fils de 13 qui est lui même devenu la racine. Enfin, on a ajouté une arête de 13 vers le représentant de 5, à savoir 4.

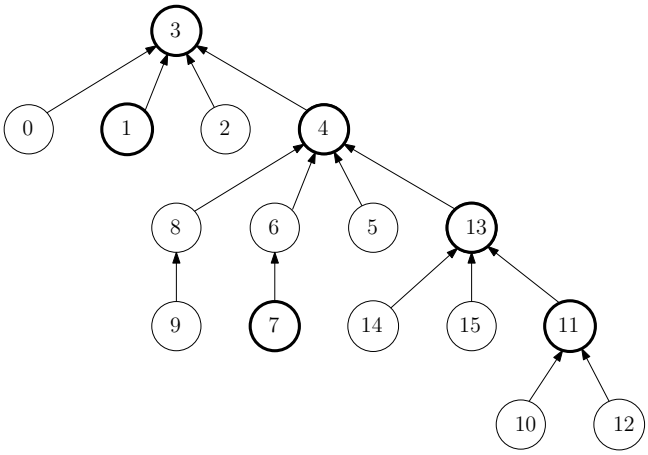


FIGURE 5 – Exemple d'ajout d'arête entre deux composantes connexes distinctes, entre les sommets 5 et 15

Question 7 : Écrire une fonction `retourner_chaine : but -> int list -> unit` qui implémente le retournement de chaîne présenté ci-dessus, sans effectuer le changement de parent de la nouvelle racine. On supposera que la liste passée en argument est une chaîne de représentants, de premier élément la racine d'un arbre.

2.2 Arêtes entre blocs distincts d'une même composante connexe

L'ajout d'une arête entre deux blocs d'une même composante connexe va entraîner la fusion de ces deux blocs ainsi que de tous les blocs compris entre les deux. Par exemple, comme illustré sur la figure **??**, l'ajout d'une arête entre les sommets 1 et 15 à partir du graphe de la figure 5 conduit à la fusion des blocs de représentants 1, 3, 4 et 13.

Question 8 : Écrire une fonction `union : but -> int -> int -> int` qui effectue l'union des blocs dont les représentants sont passés en argument. Concrètement, le parent du représentant du bloc de plus petit rang deviendra le représentant du bloc de plus grand rang. En cas d'égalité des rangs, le choix se fera de façon arbitraire, et le rang du bloc résultant augmentera de 1. Le rang d'un sommet qui n'est pas un représentant ne joue aucun rôle. On ne se souciera pas, pour le moment, de la valeur du parent du bloc obtenu. La fonction renverra le représentant du bloc final.

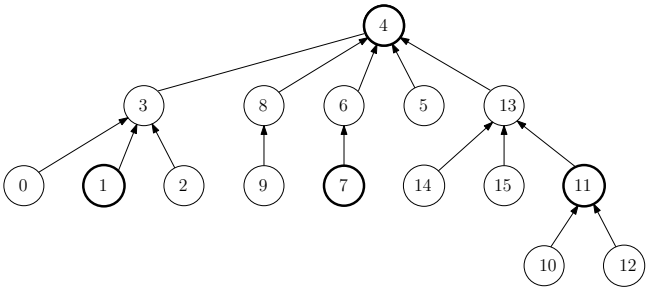


FIGURE 6 – Exemple d'ajout d'arête entre deux sommets d'une même composante connexe.

Question 9 : Écrire une fonction `fusion_chaine : but -> int list -> unit` qui effectue la fusion de tous les blocs dont les représentants sont dans la chaîne de représentants passée en argument. On portera une attention particulière, une fois la fusion de la chaîne effectuée, à la valeur du parent du représentant du bloc obtenu.

2.3 Fonction d'ajout

Question 10 : Écrire une fonction `ajout : but -> int -> int -> unit` qui implémente l'ajout d'une arête entre les deux sommets u et v passés en argument. En notant leurs représentants r_u et r_v, on distinguera alors 3 cas :

- Si ils sont égaux, u et v appartiennent au même bloc. Il n'est donc pas nécessaire de modifier la structure.
- Si ils sont différents au sein d'arbres différents, on sait qu'ils appartiennent à deux composantes connexes distinctes. On retourne alors la chaîne d'un des représentants jusqu'à la racine puis on lui attribue comme parent l'autre représentant.
- Si ils sont différents mais au sein d'un même arbre, ils appartiennent à des blocs distincts au sein d'une même composante connexe. On fusionne les blocs des chaînes reliant r_u et r_v à leur plus proche ancêtre commun qui est l'élément commun le plus profond de leurs chaînes respectives vers leur racine.

3 Calculer les blocs

On étudie enfin un graphe non orienté donné sous la forme d'un tableau de listes d'adjacence.

Question 11 : En utilisant la structure but, écrire une fonction `calcul_blocs : int list array -> int list list` qui, étant donné un graphe, renvoie la liste de ses blocs.